Placement Portal Application - Project Report

1. Author

Name: Daksh (update if required before submission)

Roll Number: BS22XXXX (update with actual roll number)

Student Email: your-email@onlinedegree.iitm.ac.in (update with actual email)

About Me: I am interested in full-stack web development and enjoy building practical systems that solve real operational problems in educational institutions.

2. AI/LLM Usage Declaration

AI Tool Used: ChatGPT (GPT family) for small assistance in writing and formatting boilerplate sections and refining UI wording.

Extent of Use: Approximately 8-12% of the overall effort (common productivity-level usage). Core logic, debugging, integration, and final validation were completed manually.

3. Project Description

This project implements a role-based Placement Portal for Admin, Company, and Student users. It replaces manual spreadsheet/email workflows with structured drive approvals, student applications, shortlisting, and history tracking.

4. Technologies Used and Purpose

Technology	Purpose
Flask	Backend routing, session handling, and controller logic
Jinja2	Server-side HTML template rendering
SQLite (sqlite3)	Local relational database and persistence
Bootstrap 5	Responsive UI structure and components
Werkzeug Security	Password hashing and credential verification
HTML/CSS	View layer and custom styling

5. DB Schema Design

The database is created programmatically in app.py (init_db). Main tables are: users, company_profiles, student_profiles, drives, and applications.

Key constraints include unique username in users, role/status check constraints, foreign keys with ON DELETE CASCADE, and UNIQUE(student_id, drive_id) to prevent duplicate applications.

Design rationale: user role data is normalized in users table, while role-specific data is separated into profile tables for cleaner extensibility and simpler access control.

6. API Design (Route-Level Design)

The application uses Flask route endpoints for each workflow: registration/login/logout, admin approvals and blacklisting, company drive operations, student application and history operations.

A separate YAML API definition file should be submitted in the final ZIP as required.

7. Architecture and Implemented Features

Code Organization: app.py contains controllers and DB helper functions; templates/ stores Jinja views; static/ contains CSS; README.md explains setup and execution.

Implemented features include: company registration approval flow, drive creation and admin approval, completion/closure behavior, student profile management, student application history, and company-side application status updates (applied/shortlisted/selected/rejected).

8. AI Percentage Check (Project-Level)

Component	Approx. Usage %	Comment
Backend logic (Flask + SQLite)	40%	Primary manual implementation effort
Templates and Styling	30%	Manual layout and iterative tuning
Testing and Debugging	20%	Manual test runs and bug fixes
AI-assisted drafting/documentation	10%	Common productivity usage only

9. Video

Drive Link (replace before final submission):
https://drive.google.com/file/d/replace-with-your-video-id/view?usp=sharing

Video length target: under 3 minutes, shared with instructors using onlinedegree account.